

Résumé × JD Analyser — Lab Guide

Note to the Reader

This lab is the practical companion to the [study material](#). You are not starting from scratch — you have a starter scaffold with the boilerplate already in place. Your job is to build the four files that contain the conceptual weight: `parse.py` , `prompts.py` , `analyzer.py` , and `main.py` .

Every task ends with a **Checkpoint** — a runnable command that confirms only that layer works before you continue. You should never be building blind.

What You'll Build

This lab produces a Python command-line tool: the **Résumé × JD Analyser** — a small-scale model of the applicant-screening pipeline a company's recruiting team might run. You supply a candidate's PDF résumé and a job description; the tool runs a multi-stage LLM pipeline and produces a structured screening report with a 0–100 score against a 60% pass threshold, indicating whether the résumé merits advancing to interview. The tool is **feedback-only** — its job is evaluation, not ghostwriting: it scores and explains, but it never rewrites the candidate's résumé.

The four pipeline stages:



Parse

`parse.py` — extract clean plain text from the résumé PDF using `pypdf` . Zero LLM calls; pure document I/O with validation.



Extract

`analyzer.py` — two LLM calls produce structured JSON profiles of both the résumé and the job description.



Analyse

`analyzer.py` — five more LLM evaluation passes: keyword match, bullet quality, terminology audit, structure check, background fit.

Step through what a complete successful run looks like:

Next Message

Play All

Replay

0 / 5 messages

Companion Reading

Keep the [study material](#) open alongside this tutorial. Each task links to the relevant section.

Task	Study material section
Task 1: <code>parse.py</code>	§5 Document Parsing
Task 2: <code>llm.py</code>	§8 Handling Imperfect AI Output
Task 3: <code>prompts.py</code>	§3.3 Schema-First Prompt Design · §6.1 Extraction Prompts · §6.2 Evaluation Prompts · §6.3 Feedback-Only Principle
Task 4: <code>analyzer.py</code>	§4 The Multi-Stage Pipeline · §7.2 Weighted Aggregation
Task 5: <code>main.py</code>	§4.3 Data Flow Between Stages

Prerequisites

- Comfort with LLM completion calls (e.g. via LiteLLM or an equivalent SDK), the `messages` array, `json.loads` / `json.dumps` , `.env` secrets management, and the ICCO prompt-design framework (Instruction, Context, Constraints, Output).
- Python 3.11 or later installed.
- An API key for OpenAI **or** Anthropic **or** a locally running Ollama model.

Task 0: Setup

Duration: ~15 minutes

Five steps to get the project running before you write a single line of analysis code:

Step 1 — Copy the starter

Copy the `day4_starter_track_a/` folder to a new folder named `day4_project/` , then open that folder in your terminal:

Platform	How to copy
File manager (any OS)	Right-click <code>day4_starter_track_a/</code> → Copy, paste alongside it, rename the copy to <code>day4_project</code>
macOS / Linux terminal	<code>cp -r day4_starter_track_a day4_project</code>
Windows terminal	<code>xcopy /E /I day4_starter_track_a day4_project</code>

What you'll find in your project folder:

-  `day4_project/` Root – your working copy of the project
 -  `.env.example` Template – copy to `.env` and add your API key (Step 4)
 -  `.gitignore` Keeps `.env` and `outputs/` out of git
 -  `requirements.txt` Three packages: `litellm`, `python-dotenv`, `pypdf`
 -  `llm.py` Pre-provided – the LLM abstraction layer (Task 2: read it, do not modify)

-  `report.py` Pre-provided – renders the analysis results dict into a Markdown report file
-  `parse.py` You build this – Task 1
-  `prompts.py` You build this – Task 3
-  `analyzer.py` You build this – Task 4
-  `main.py` You build this – Task 5 (Track A) / Pre-provided (Track B)
-  **inputs/** Pre-provided sample inputs – all files already present in the starter
 -  `strong_resume.pdf` Pre-provided – strong résumé sample; primary input for Tasks 1–5
 -  `weak_resume.pdf` Pre-provided – weak résumé sample; used in Final Verification contrast runs
 -  `job_rtis_systems_engineer.txt` Pre-provided – the primary RTIS job description fixture
 -  `job_unrelated.txt` Pre-provided – unrelated job for Final Verification contrast runs
-  **outputs/** Reports land here – one `.json` and one `.md` per run
 -  (empty until Task 5)

Step 2 — Create a virtual environment

```
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
```

Step 3 — Install dependencies

```
pip install -r requirements.txt
```

This installs three packages: `litellm` , `python-dotenv` , and `pypdf` .

Step 4 — Configure your API key

Duplicate `.env.example` and name the copy `.env` (both files sit in the same `day4_project/` folder). Then open `.env` in a text editor and fill in your key. The defaults work with a local Ollama model — no API key needed:

```
MODEL=ollama/gemma4:e2b
OLLAMA_API_BASE=http://localhost:11434
```

Run `ollama serve` in a separate terminal and pull the model first with `ollama pull gemma4:e2b`.

To use OpenAI instead:

```
MODEL=openai/gpt-4o-mini
OPENAI_API_KEY=sk-...
```

To use Anthropic Claude instead:

```
MODEL=anthropic/claude-3-5-sonnet-20241022
ANTHROPIC_API_KEY=sk-ant-...
```

Step 5 — Verify sample inputs

The starter pack already includes all résumé and job description files. Open the `inputs/` folder and confirm these four files are present:

- `strong_resume.pdf` — strong résumé sample (primary input for Tasks 1–5)
- `weak_resume.pdf` — weak résumé sample (used in Final Verification)
- `job_rtis_systems_engineer.txt` — RTIS job description fixture
- `job_unrelated.txt` — unrelated job description fixture

Note: Both PDFs are text-based exports — `pypdf` can extract their content directly. You do not need to supply your own résumé PDF to complete this lab.

✓ Checkpoint

Run:

```
python -c "from llm import ask_json; print('LLM layer ready')"
```

What success looks like:

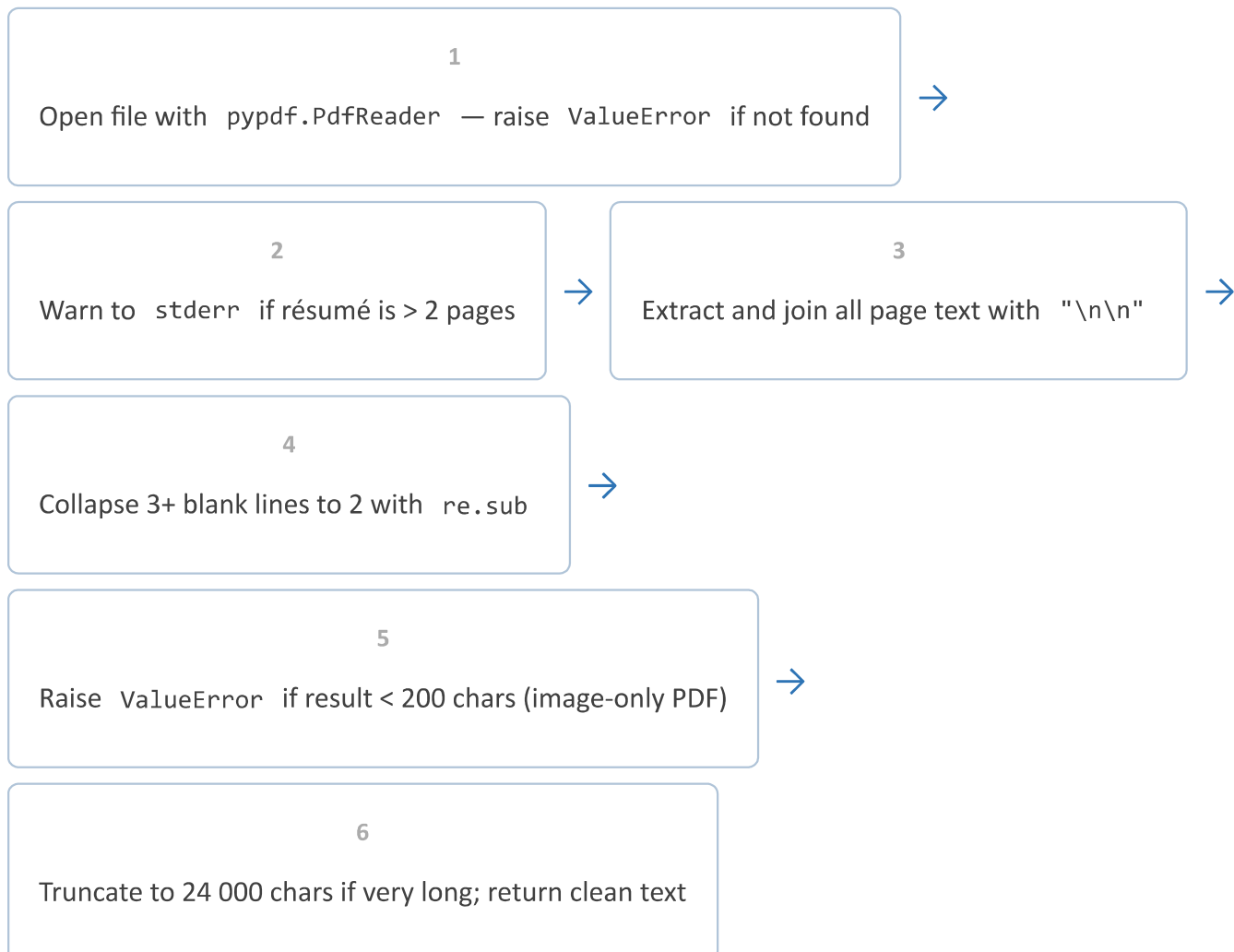
- The shell prints `LLM layer ready` with no traceback
- No `ModuleNotFoundError` — if one appears, re-run `pip install -r requirements.txt` inside the active `virtualenv`

Task 1: Parse the Résumé

Study material: [§5 Document Parsing](#)

What this module does: `parse.py` reads a PDF résumé file and returns clean plain text. It also reads plain-text job description files. This stage contains **no LLM calls** — it is pure document I/O with validation. The study material explains why we validate inputs *before* passing them to the LLM ([§5: the validate-before-LLM rule](#)).

How `read_resume_pdf` processes a file — every step in order:



You need to implement two functions:

```
def read_resume_pdf(path: str) -> str: ...

def read_jd_text(path: str) -> str: ...
```

Open `parse.py` in your editor. You are **creating this file** in Task 1. The starter includes docstrings and function signatures — implement the two function bodies below.

Open `parse.py` . The functions have full docstrings — implement them directly.

`read_resume_pdf(path)` requirements:

1. Open the file with `pypdf.PdfReader(path)` ; raise `ValueError` with a helpful message if the file is not found or cannot be opened.
2. If the résumé has more than 2 pages, print a warning to `sys.stderr` (ATS typically expects a one-page résumé).
3. Extract text from each page with `page.extract_text()` or `""` , then join pages with `"\n\n"` .
4. Collapse runs of 3+ blank lines to 2 with `re.sub(r"\n{3,}", "\n\n", text)` .
5. Raise `ValueError` if the result has fewer than 200 characters (likely an image-only PDF).
6. Truncate to 24 000 characters if very long (with a warning to `sys.stderr`).

`read_jd_text(path)` requirements:

1. Open the file with `open(path, encoding="utf-8")` .
2. Raise `ValueError` with a helpful message if the file is not found.
3. Raise `ValueError` if the stripped content has fewer than 100 characters.

✓ Checkpoint

Run:

```
python -c "
from parse import read_resume_pdf
text = read_resume_pdf('inputs/strong_resume.pdf')
print(f'Extracted {len(text)} chars')
print(text[:200])
"
```


What success looks like:

- Character count above 200
- The candidate's name visible in the first 200 characters of output

Also test the error case:

```
python -c "  
from parse import read_resume_pdf  
read_resume_pdf('inputs/does_not_exist.pdf')  
" 2>&1
```

What success looks like:

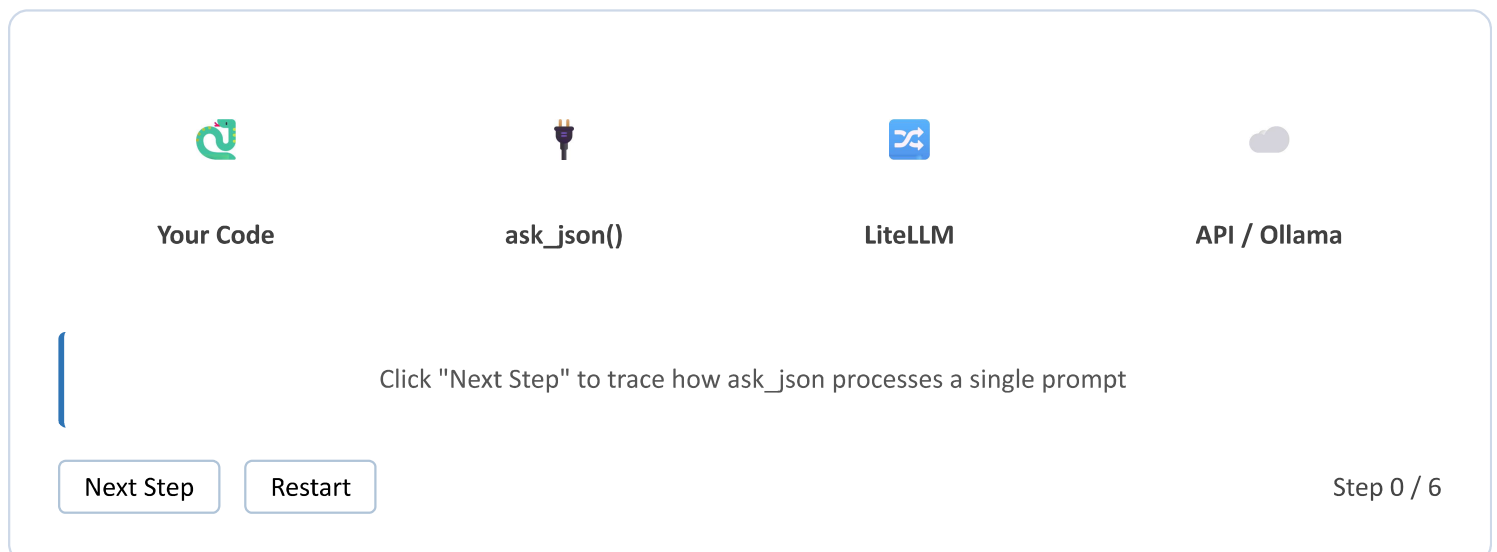
- A `ValueError` with a message like `Résumé file not found: inputs/does_not_exist.pdf`

Task 2: The LLM Layer — Read and Run

Study material: [§8 Handling Imperfect AI Output](#)

What this task is: `llm.py` is pre-provided. You are **not** building it — you are reading and verifying it. You need to understand what `ask_json()` does before you use it in the next two tasks, because the quality of your prompts (and your descriptions to Claude) depends on knowing what the layer beneath them does.

Trace how `ask_json` moves a prompt through the stack:



Open `llm.py` in your editor. This file is **pre-provided — do not modify it**. Read through it to understand `ask_json()` before you call it in Tasks 3 and 4.

Skim `llm.py` and answer these three questions in your head before continuing:

1. Which private function strips markdown fences (````json`) from the LLM's response?
2. What does `json.JSONDecoder().raw_decode(text, start)` do that `json.loads(text)` does not? Why does this matter when a model adds a preamble sentence before the JSON?
3. What is `_check_no_rewrite()` and what does it detect?

✓ Checkpoint

Run:

```
python -c "  
from llm import ask_json  
result = ask_json('Return {"status": "ok"} and nothing else.', 'go')  
print(result)  
"
```

What success looks like:

- `{'status': 'ok'}` printed to the terminal
- One complete round-trip worked: the model returned valid JSON and `ask_json` parsed it cleanly

If you see an API error, recheck your `.env` file and API key.

Task 3: Write the Prompts

Study material: [§3.3 Schema-First Prompt Design](#) · [§6.1 Extraction Prompts](#) · [§6.2 Evaluation Prompts](#) · [§6.3 Feedback-Only Principle](#)

This is the conceptual heart of the lab. Writing these system prompts applies the ICCO framework — Instruction, Context, Constraints, Output — to automated résumé analysis instead of a back-and-forth chat conversation. The same four components structure every effective prompt here.

The eight prompts fall into three categories — each calls a different approach:

The eight prompt constants you need to write:

Constant	Type	Purpose	Key output field(s)
RESUME_PROFILE_PROMPT	Extraction	Candidate profile from résumé text	name , skills , projects , experience
JD_PROFILE_PROMPT	Extraction	Role requirements from JD text	required_skills , preferred_skills
KEYWORD_MATCH_PROMPT	Evaluation	Compare résumé keywords vs JD keywords	keyword_match_score , missing
BULLET_QUALITY_PROMPT	Evaluation	Score each bullet on ATI rubric	bullet_quality_avg , bullets
JARGON_AUDIT_PROMPT	Evaluation	Flag résumé/JD terminology mismatches	jargon_score , flags
STRUCTURE_AUDIT_PROMPT	Evaluation	Check ATS-parseability formatting	structure_score
BACKGROUND_FIT_PROMPT	Evaluation	Candidate background fit for the role	background_fit_score
OVERALL_SUMMARY_PROMPT	Synthesis	3-bullet executive summary	plain text (not JSON)

Every prompt must: - Follow ICCO structure (Instruction → Context → Constraints → Output) - Embed the exact JSON schema in the Output section (except `OVERALL_SUMMARY_PROMPT`) - End with this exact line: "Output ONLY a valid JSON object matching the schema above. No prose. No markdown fences. No commentary. Never rewrite or generate résumé content." - Use temperature `0.0 – 0.1` for extraction prompts, `0.2 – 0.3` for evaluation prompts

Open `prompts.py` in your editor. You are **creating this file** in Task 3. Add the eight string constants described below. Each becomes a system prompt fed to `ask_json()`.

Important — no placeholders in the prompt constants. All eight constants are plain static strings — no f-strings, no `.format()`, no `{variable}` slots. The `{` and `}` characters you see inside the prompts are **literal JSON schema syntax**, not Python format placeholders. Dynamic data (the résumé text, JD text, degree code) is never embedded in the system prompt; `analyzer.py` passes it as the second argument to `ask_json()` — the user message. Your job in this task is only to write the static instruction text.

Open `prompts.py`. The schema comments show the expected JSON structure for each constant. Write the full ICCO-structured string for each one.

For the extraction prompts (`RESUME_PROFILE_PROMPT` , `JD_PROFILE_PROMPT`): the Instruction says "extract only what is literally present — never invent". The Output section embeds the full JSON schema verbatim.

For the evaluation prompts (`BULLET_QUALITY_PROMPT` , `JARGON_AUDIT_PROMPT` , `STRUCTURE_AUDIT_PROMPT` , `BACKGROUND_FIT_PROMPT`): the Context section grounds the model in well-known, general résumé/ATS conventions — no external reference document is needed. The Constraints section must state that `what_is_missing` , `suggested_translation` , and similar diagnostic fields must never become rewrites.

Key rubrics to write from first principles: - **BULLET_QUALITY_PROMPT** — the Action → Technology → Impact rubric: a strong bullet names the action taken, the specific technology/tool used, and a measurable impact. Define L1 (action only), L2 (action + technology), L3 (action + technology + impact). - **JARGON_AUDIT_PROMPT** — no static table: instruct the model to dynamically compare résumé terminology against JD terminology and flag likely-equivalent-but-differently-worded terms, plus the severity scoring rules (`jargon_score = max(0, 100 - 10*high - 5*medium - 2*low)`) - **STRUCTURE_AUDIT_PROMPT** — general ATS-parseability rules: single-column layout, standard section headers, reverse-chronological order, appropriate length, contact info placement, no images/graphics - **BACKGROUND_FIT_PROMPT** — judge alignment using only `resume_profile` 's education/experience fields and `jd_profile` 's requirements — no external degree code or table

`OVERALL_SUMMARY_PROMPT` returns plain Markdown bullets, not JSON — omit the JSON Output constraint for this one.

✓ Checkpoint — Résumé Extraction

Run:

```
python -c "
from parse import read_resume_pdf
from llm import ask_json
from prompts import RESUME_PROFILE_PROMPT
text = read_resume_pdf('inputs/strong_resume.pdf')
result = ask_json(RESUME_PROFILE_PROMPT, f'RÉSUMÉ TEXT:\n\n{text}', max_tokens=2000)
print('Name:', result.get('name'))
print('Languages:', result.get('skills', {}).get('languages', []))
"
```

What success looks like:

- The candidate's real name printed

- A non-empty list of programming languages extracted from the résumé

If you see empty fields: review the extraction constraints in `RESUME_PROFILE_PROMPT`. The "If a field is absent, return empty array" rule sometimes causes models to return empty arrays even when data is present — tighten the extraction instruction.

✓ Checkpoint — Keyword Match

Run:

```
python -c "
from parse import read_resume_pdf, read_jd_text
from llm import ask_json
from prompts import RESUME_PROFILE_PROMPT, JD_PROFILE_PROMPT, KEYWORD_MATCH_PROMPT
import json

resume = read_resume_pdf('inputs/strong_resume.pdf')
jd = read_jd_text('inputs/job_rtis_systems_engineer.txt')
rp = ask_json(RESUME_PROFILE_PROMPT, f'RÉSUMÉ TEXT:\n\n{resume}', max_tokens=2000)
jp = ask_json(JD_PROFILE_PROMPT, f'JOB DESCRIPTION TEXT:\n\n{jd}', max_tokens=1500)
km = ask_json(KEYWORD_MATCH_PROMPT,
              f'RÉSUMÉ PROFILE:\n\n{json.dumps(rp, indent=2)}\n\nJD PROFILE:\n\n{json.dumps(jp, indent=2)}',
              max_tokens=3000)
print('Score:', km.get('keyword_match_score'))
print('Missing (first 3):', km.get('missing', [])[:3])
"
```

What success looks like:

- A score between 0 and 100
- A list of missing keywords (can be empty if your résumé is a strong match)

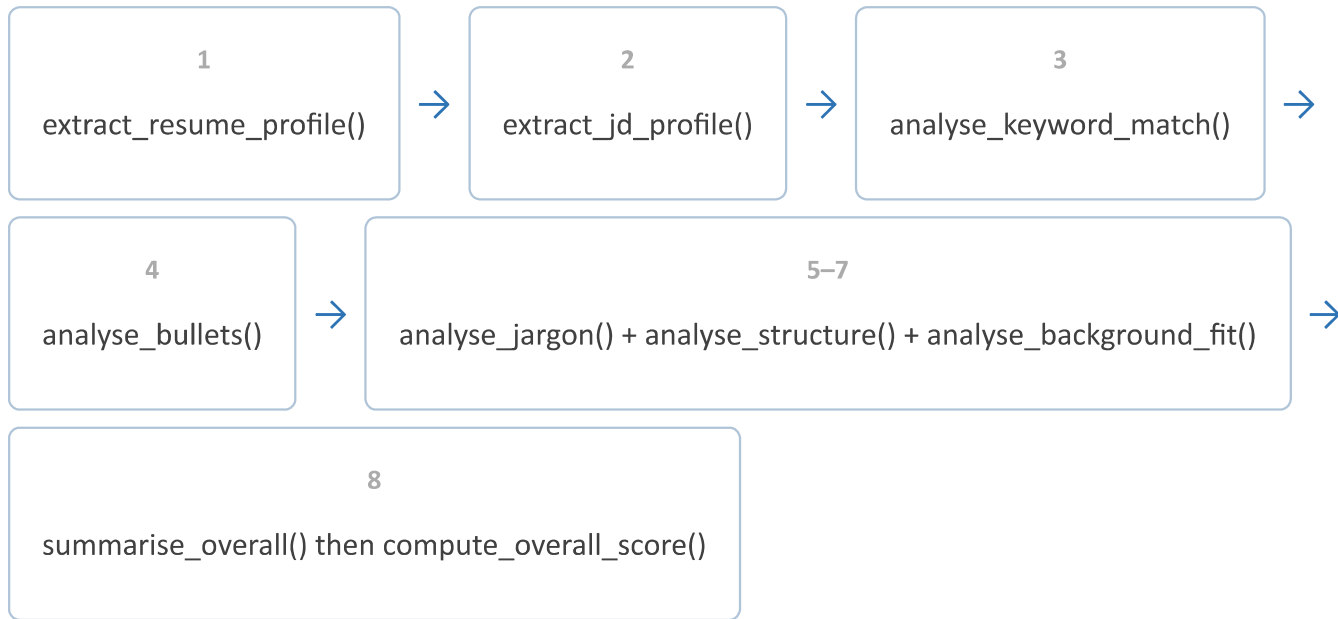
Task 4: Build the Analysis Pipeline

Study material: [§4 The Multi-Stage Pipeline](#) · [§7.2 Weighted Aggregation](#)

What this module does: `analyzer.py` contains 8 analysis functions that each wrap exactly one LLM call, plus one pure-Python scoring function (`compute_overall_score`) with no LLM call at all.

The [study material §4.2](#) says: *smoke-test each layer; integrate after each layer is verified*. This task practices that rule — the Checkpoint tests a subset of functions without requiring the rest to exist yet.

All eight functions — in the order `main.py` calls them:



Open `analyzer.py` in your editor. You are **creating this file** in Task 4. Function signatures and docstrings are already present in the starter — implement each function body below.

Open `analyzer.py` . Function signatures and docstrings are provided; implement each one.

Key implementation notes:

- `ask_json(system, user, max_tokens=N)` — the system is the prompt constant; the user is the formatted input data (JSON-dumped where needed).
- `analyse_jargon()` takes two positional arguments: `resume_profile` , `jd_profile` .
- `summarise_overall()` uses `ask_text()` , not `ask_json()` — it returns a plain string.
- `compute_overall_score()` must use these exact weights (no LLM call):

Field in report	Key	Weight
report['keyword_match']['keyword_match_score']	keyword match	0.40
report['bullets']['bullet_quality_avg']	bullet quality	0.25
report['structure']['structure_score']	structure	0.15
report['jargon']['jargon_score']	jargon	0.10
report['background_fit']['background_fit_score']	background fit	0.10

Return int(round(total)) .

✓ Checkpoint

Run:

```
python -c "
from parse import read_resume_pdf, read_jd_text
from analyzer import extract_resume_profile, extract_jd_profile, analyse_keyword_match, compute
import json
resume = read_resume_pdf('inputs/strong_resume.pdf')
jd = read_jd_text('inputs/job_rtis_systems_engineer.txt')
rp = extract_resume_profile(resume)
jp = extract_jd_profile(jd)
km = analyse_keyword_match(rp, jp)
fake_report = {
    'keyword_match': km,
    'bullets': {'bullet_quality_avg': 60},
    'structure': {'structure_score': 70},
    'jargon': {'jargon_score': 90},
    'background_fit': {'background_fit_score': 80},
}
print('Composite score:', compute_overall_score(fake_report))
"
```

What success looks like:

- Terminal prints Composite score: followed by a number between 0 and 100 (e.g. Composite score: 63)
- No traceback — this exercises extract_resume_profile , extract_jd_profile , analyse_keyword_match , and compute_overall_score without requiring the remaining functions yet

Task 5: Wire the Pipeline

Study material: [§4.3 Data Flow Between Stages](#)

Open `main.py` in your editor. You are **implementing the `main()` function** in Task 5. The `parse_args()` helper is already present — write the `main()` function body below it.

What to implement: The pipeline must:

1. Call `parse_args(sys.argv)` to get `resume_path` , `job_path` .
2. Print `Using model: {model}` where `model` comes from the `MODEL` env var.
3. Call `read_resume_pdf()` and `read_jd_text()` with `try/except ValueError` — print the error to `sys.stderr` and return `1` on failure.
4. Call all 8 analysis functions in sequence. For each step print `[N/8] <description>` . Wrap the LLM calls in `try/except RuntimeError` — print and return `1` on failure.
5. Build the `report` dict with keys: `meta` , `resume_profile` , `jd_profile` , `keyword_match` , `bullets` , `jargon` , `structure` , `background_fit` .
6. Call `compute_overall_score(report)` and set `report["passes_ats_threshold"]` .
7. Call `summarise_overall(report)` and set `report["summary"]` .
8. Write `outputs/match_report_{timestamp}.json` (use `json.dumps`) and `outputs/match_report_{timestamp}.md` (use `render_markdown(report, out_path=md_path)` — import it from `report.py` with `from report import render_markdown`).
9. Print the final score, verdict, file paths, and summary bullets.

Checkpoint

Run:

```
python main.py inputs/strong_resume.pdf inputs/job_rtis_systems_engineer.txt
```

What success looks like:

```
Using model: ollama/gemma4:e2b
[1/8] Parsing résumé: inputs/strong_resume.pdf
[2/8] Reading JD: inputs/job_rtis_systems_engineer.txt
```

```
[3/8] Extracting résumé profile (LLM)...
[4/8] Extracting JD profile (LLM)...
[5/8] Keyword match (LLM)...
[6/8] Bullet audit (LLM)...
[7/8] Jargon, structure, background fit (LLM x3)...
[8/8] Final summary (LLM)...
```

Score: XX/100 (PASS 60% ATS threshold)

JSON: outputs/match_report_YYYYMMDD_HHMMSS.json

MD: outputs/match_report_YYYYMMDD_HHMMSS.md

- <executive summary bullet 1>
- <executive summary bullet 2>
- <executive summary bullet 3>

Score below 60 shows FAIL instead of PASS. Two report files should appear in outputs/.

Final Verification — All Four Combinations

You have built a working AI analysis pipeline. These four runs are a designed experiment: each combination holds one variable constant while changing the other. Your goal is not to hit a score target — it is to watch your tool produce different outputs from the same code and understand *why*.

There is no grading and no submission. Open the .md report from outputs/ after each run and read what your system produced.

Run 1 – Best case: strong résumé, relevant job

```
python main.py inputs/strong_resume.pdf inputs/job_rtis_systems_engineer.txt
```

Run 2 – Weak résumé, same relevant job (isolates résumé quality)

```
python main.py inputs/weak_resume.pdf inputs/job_rtis_systems_engineer.txt
```

Run 3 – Strong résumé, wrong job (isolates job fit)

```
python main.py inputs/strong_resume.pdf inputs/job_unrelated.txt
```

Run 4 – Worst case: both variables fail

```
python main.py inputs/weak_resume.pdf inputs/job_unrelated.txt
```

After each run, open the .md report and check the **Score Breakdown** table at the bottom — it shows exactly which of the five components pulled the total up or down.

Run	Expected verdict	What to look for in the .md report
1: strong + relevant job	PASS (65+)	High <code>keyword_match_score</code> ; long "present" keyword list; few or no terminology flags
2: weak + relevant job	FAIL (35–55)	Long <code>missing</code> keyword list with many <code>required</code> items; Bullet Quality Audit shows mostly L1/L2; compare Score Breakdown with Run 1 row by row
3: strong + unrelated	FAIL (20–45)	<code>keyword_match_score</code> collapses; <code>bullet_quality_avg</code> is nearly identical to Run 1 — only one thing changed
4: weak + unrelated	FAIL (10–30)	Every Score Breakdown row is low; this is the floor — use it as a reference when reading Runs 1–3

Reflection

These are not graded questions. They are prompts for your own reasoning — worth thinking through now, before you extend this project further.

1. Quality versus relevance

Place the Score Breakdown tables from Run 1 and Run 3 side by side (open both .md reports). The résumé is identical in both runs.

- Which rows in the breakdown changed between the two?
- Which rows stayed the same?
- The score difference between Run 1 and Run 3 is driven almost entirely by one component. Which one, and what does that tell you about what an ATS actually measures?

2. What the weak résumé is missing

Open the Run 2 .md report (weak résumé + relevant job). In the Keyword Match section, find the first three `required` keywords listed under "missing."

- Are those skills the candidate never learned — or skills they likely have but did not name in the résumé?
- If it is the latter, what does that imply about the value of keyword visibility versus actual skill level?

3. Inside a bullet

Open the Bullet Quality Audit in any run. Find one bullet rated L1 or L2. Read its `what_is_missing` field.

- Which of the three ingredients (Action, Technology, Impact) is absent?
- Without rewriting the bullet text, describe in one sentence what specific information would push it to L3.

4. The 40% weight

`keyword_match_score` carries 40% of the total — the largest single weight. Your Run 3 shows a strong résumé failing because of keyword mismatch alone.

- Why might a real ATS weight keyword density so heavily relative to writing quality?
 - Is that weighting fair to a candidate with strong work who is switching industries?
 - If you were redesigning the weights for a design-focused role (e.g. UX/UI design) rather than a systems-engineering role, what would you change?
-

What You Built

Your pipeline ran on all four combinations and produced eight report files. You implemented:

- A PDF parser that validates inputs before any LLM call
- Eight prompt constants grounding evaluation in well-known, general résumé/ATS and hiring conventions
- An eight-call LLM pipeline with structured JSON output at every stage
- A weighted scoring formula and a human-readable Markdown report

The `.md` reports in `outputs/` are real, grounded screening feedback — produced by the system you wrote, in the same shape a company's own applicant-screening pipeline would produce.